

Luego de realizado el ejercicio agregar al código para la posibilidad de cargar 50000 clientes y 50000 artículos. ¿Qué campos considera el Equipo que pueden considerarse índices y dar un ejemplo de consulta en JPAQL utilizando esos campos índices y consulta sin utilizar índice? Analizar el tiempo de respuesta y describir las conclusiones que obtuvo el equipo.

¿Cuál sentencia de SQL, teniendo en cuenta la base de datos utilizada, el Equipo puede tener en cuenta para conocer el costo (tiempo de respuesta) que tiene una consulta a la base de datos?

Los campos que consideramos índice de la clase Cliente son el id, ya que es su identificador único en la base de datos, y el DNI, ya que también es un identificador único de cada cliente y es mucho más probable que quien use el sistema consulte por DNI del cliente que por el id que le asignó a ese cliente la base de datos.

Y en Producto (Artículo) creemos que sus índices son id y denominación, por los mismos motivos que lo son id y DNI en cliente.

```
@Table(name = "articulo", indexes = {
    @Index(name = "idx_id_art", columnList = "id"),
    @Index(name = "idx_denom_art", columnList = "denominacion")
})
```

```
@Table(name = "cliente", indexes = {
    @Index(name = "idx_id_cliente", columnList = "id"),
    @Index(name = "idx_cliente_dni", columnList = "dni")
})
```

Ahora probamos los índices y comparamos (explain analyze nos dice cuánto tiempo tardó la consulta en ejecutar).

Consulta en Cliente sobre dni:

```
EXPLAIN ANALYZE SELECT * FROM cliente WHERE dni = 30021466;
```

- Con índice:

```
EXPLAIN ANALYZE SELECT * FROM cliente WHERE dni = 30021466;
```

PLAN

SELECT

```
"PUBLIC"."CLIENTE"."ID",
"PUBLIC"."CLIENTE"."APELLIDO",
"PUBLIC"."CLIENTE"."DNI",
"PUBLIC"."CLIENTE"."NOMBRE",
"PUBLIC"."CLIENTE"."FK_DOMICILIO"
```

FROM "PUBLIC"."CLIENTE"

```
/* PUBLIC.IDX_CLIENTE_DNI: DNI = 30021466 */
```

```
/* scanCount: 2 */
```

```
WHERE "DNI" = 30021466
```

(1 row, 5 ms)

Tarda 5 ms

- Sin índice:

```
EXPLAIN ANALYZE SELECT * FROM cliente WHERE dni = 30021466;
```

PLAN

```
SELECT
  "PUBLIC"."CLIENTE"."ID",
  "PUBLIC"."CLIENTE"."APELLIDO",
  "PUBLIC"."CLIENTE"."DNI",
  "PUBLIC"."CLIENTE"."NOMBRE",
  "PUBLIC"."CLIENTE"."FK_DOMICILIO"
FROM "PUBLIC"."CLIENTE"
/* PUBLIC.CLIENTE.tableScan */
/* scanCount: 50001 */
WHERE "DNI" = 30021466
```

(1 row, 48 ms)

Tarda 48 ms

Consulta en Artículo sobre denominacion:

```
EXPLAIN ANALYZE SELECT * FROM articulo WHERE denominacion = 'generico 37921';
```

- Con índice:

```
EXPLAIN ANALYZE SELECT * FROM articulo WHERE denominacion = 'generico 37921';
```

PLAN

```
SELECT
  "PUBLIC"."ARTICULO"."ID",
  "PUBLIC"."ARTICULO"."CANTIDAD",
  "PUBLIC"."ARTICULO"."DENOMINACION",
  "PUBLIC"."ARTICULO"."PRECIO"
FROM "PUBLIC"."ARTICULO"
/* PUBLIC.IDX_DENOM_ART: DENOMINACION = 'generico 37921' */
/* scanCount: 1 */
WHERE "DENOMINACION" = 'generico 37921'
```

(1 row, 2 ms)

2ms

- Sin índice:

```
EXPLAIN ANALYZE SELECT * FROM articulo WHERE denominacion = 'generico 37921';
```

PLAN

```
SELECT
  "PUBLIC"."ARTICULO"."ID",
  "PUBLIC"."ARTICULO"."CANTIDAD",
  "PUBLIC"."ARTICULO"."DENOMINACION",
  "PUBLIC"."ARTICULO"."PRECIO"
FROM "PUBLIC"."ARTICULO"
/* PUBLIC.ARTICULO.tableScan */
/* scanCount: 50001 */
WHERE "DENOMINACION" = 'generico 37921'
```

```
(1 row, 55 ms)
```

55ms

¿Cuál es la configuración en JPA que se puede utilizar para conocer cómo el motor de persistencia transforma los JPQL en sentencia SQL? Dar un ejemplo del JPAQL para la obtención de Factura y como queda esa sentencia expresada en SQL.

El ORM es el que se encarga de las consultas SQL para que nosotros no tengamos que hacer queries y en el código tengamos únicamente métodos del ORM (como find) que ejecutan consultas de forma transparente al programador.

Para que por consola aparezcan esas traducciones a SQL que hace el ORM (en nuestro caso hibernate) podemos configurarlo en el archivo persistence.xml con la línea:

```
<property name="hibernate.show_sql" value="true"/>
```

Así por ejemplo podemos buscar en nuestra base de datos una factura que hayamos persistido previamente de esta manera:

Sentencia JPAQL:

```
SELECT f FROM Factura f WHERE f.id = 3;
```

Traducido a SQL:

```
SELECT f1_0.id, f1_0.fecha, f1_0.total, f1_0.cliente_id, f1_0.forma_pago_id
FROM factura f1_0
WHERE f1_0.id = 3;
```